

ARC-AGI-3 in five experiments

ARC-AGI-3 is a set of puzzle games with no instructions: a screen of coloured squares, a few buttons, and you work out the goal by trying things. We asked whether a small, free AI model — helped by a smart program around it — could play them better than pressing random buttons. Here is each experiment, told the same way.

Every experiment had its pass/fail rule written down before it ran. Numbers are from 17 test games the AI never practised on, unless marked “practice games”. [How we tested, and why the games were split.](#)

E0

Just let the AI play

■ NO BETTER
THAN RANDOM

E1

Give it a memory

■ WEAK

E2

Explain the buttons first

■ MADE IT
WORSE

E3

Label moves: learn or win

■ MADE IT
WORSE

E4

Smarter random, no AI

■ WORKED,
THEN DIDN'T

E0 · THE STARTING POINT

■ NO BETTER THAN RANDOM

Just let the AI play

Before adding anything clever, see how the plain AI does.

THE QUESTION

Does a small AI, choosing every move itself, beat random button-pressing?

WHAT WE DID

Showed the AI the screen as text and let it pick a button each turn, 80 moves per game, 5 tries per game. We tried a small model and one 8× bigger.

WHAT HAPPENED

0

levels cleared, small AI

0

levels cleared, big AI

1

level cleared, random

The AI handled arrow keys fine but kept **clicking squares that do nothing**. The bigger model wasted fewer clicks, but still cleared nothing.

WHAT IT MEANS

A small AI on its own is not better than chance here. Its main weakness is clicking the wrong things.

▼ A bit more detail

Models: Gemma-4-E4B and Gemma-4-31B. Levels per game: random 0.0125, both models 0.000. Exploration (different screens seen per 100 moves): random 78.5, small AI 68.6, big AI 73.2.

A third model, Qwen3-8B, kept choosing buttons the games did not have, so its run tells us about our setup, not about how well it plays.

E1 • MEMORY

■ WEAK

Give the AI a memory

If the AI remembers what didn't work, it should stop repeating mistakes.

THE QUESTION

Does memory help the AI beat both its old self and random play?

WHAT WE DID

Added two helpers, and tested them alone and together:

Block — if a button did nothing on this screen, don't let the AI press it again here.

Notes — after each game over, write down what happened just before.

WHAT HAPPENED

1

level cleared, both helpers

1

level cleared, random

68.6 → 74.4

exploration with blocking

Blocking made the AI explore more. The notes did nothing. The one level cleared came mostly from the helper swapping in random moves.

WHAT IT MEANS

Memory helps the AI explore, but not win. It tied random; it needed to beat it.

▼ A bit more detail

In the clearing game, 64% of moves were the helper's random swaps. The notes helper wrote 34 notes, but the AI rarely died the same way twice, so there was little to fix — this is not strong evidence that notes are useless.

E2 • BUTTON CHECK

■ MADE IT WORSE

Explain the buttons first

If the AI knows what each button does before playing, it should waste fewer moves.

THE QUESTION

Does a short “what each button does” list help the AI stop wasting moves?

WHAT WE DID

A helper pressed a few buttons first and wrote down what changed. The AI then played with that list. For a fair comparison, a second AI got the same number of extra moves but no list. Tested on practice games first.

WHAT HAPPENED

34%

wasted moves, with the list

28%

wasted moves, without

With the list, the AI wasted **more** moves. On one game, wasted moves roughly doubled. We stopped before the real test, as our rule said.

WHAT IT MEANS

Telling this AI what buttons do didn't help it. We can't tell if the idea is wrong or our version was too crude.

▼ A bit more detail

Shown for the helper pressing 5 buttons first. At 10 and 20 buttons the list still lost: 32% vs 30%, and 33% vs 29%. On some games the list did make the AI explore more — just not in the way we needed.

E3 • LEARN OR WIN

■ MADE IT WORSE

Label each move: to learn, or to win

A good player mixes “let me try this to find out” moves with “this should get me closer to the goal” moves.

THE QUESTION

Would the AI play better if it marked each move as a learning move or a winning move, and kept a diary of what its learning moves did?

WHAT WE DID

The AI had to start each reply with **LEARN** or **WIN**. After every LEARN move, the helper wrote down what changed on screen, and the AI saw its last five “experiments”. Unlike E2, the AI chose what to test. Tested on practice games first, with the same rule as E2.

WHAT HAPPENED

34%

wasted moves, with labels

30%

wasted moves, without

2 in 3

replies with no label

With labels the AI wasted **more** moves, and mostly **ignored the instruction**. We stopped at the practice stage, as our rule said.

WHAT IT MEANS

This AI doesn't reliably follow the labelling, and where it did, it played no better. That's a weak test — not proof the idea is bad.

▼ A bit more detail

We had set this idea aside at the start, because other teams have already published it on this benchmark (for example Tycho's "active abstraction" and OPINE-World). We ran it anyway to close the question. Of 1,920 replies, 385 said LEARN, 255 said WIN and 1,280 had no label. The same game as in E2 (sk48) carried the loss. Cost: about 1 GPU-hour.

E4 • SMARTER RANDOM

■ WORKED, THEN DIDN'T

Smarter random, no AI at all

If random play is the thing to beat, make random play a little smarter.

THE QUESTION

Does random button-pressing clear more levels if it never repeats a button it already saw do nothing?

WHAT WE DID

Two random players — one plain, one that skips known do-nothing buttons — on the same games, 300 moves each, 20 tries per game. It ran on a laptop, no AI needed. Then we entered the smarter one in the competition.

WHAT HAPPENED

38 vs 19

levels cleared, smarter vs plain

18 of 20

tries won, none lost

0.07 → 0.07

competition score

Smarter random **doubled** the levels cleared — but almost all of it came from **one game**. On the competition's 110 hidden games, the score did not change at all.

WHAT IT MEANS

A real effect on the games we tested, but it did not carry over to new games.

▼ A bit more detail

The one game is lp85: smarter random cleared it in 20/20 tries, plain random in 1/20. Everywhere else they cleared the same levels. The competition shows scores to two decimals, so a very small change would not be visible.

HOW WE TESTED

Practice games, test games, and the steps in between

Every experiment on this page went through the same process. That process matters as much as the results, because it decides which results can be trusted.

The games were split once, before any AI played

There are 25 public games. We shuffled them with a fixed number (20260912) so the split could not be nudged, and put 8 aside for **practice** and 17 aside for the **test**. The split has never changed.

8 practice games “tune” — try settings, find bugs, decide whether an idea is worth testing

dc22 ft09 ls20 s5i5 sc25 sk48 tn36 tu93

17 test games “held-out” — kept unseen until the verdict run; only these decide a result

ar25 bp35 cd82 cn04 g50t ka59 lf52 lp85 m0r0 r11l re86 sb26
sp80 su15 tr87 vc33 wa30

r11l (dashed) is reported but left out of the totals: its first level is so easy that random button-pressing can beat a human's time, so it would reward luck. That leaves **16 games** in every verdict.

110 hidden games the competition's own test — nobody outside the organisers can see them



Like a practice exam and a final exam

If you adjust an idea on the same games you grade it on, you end up fitting those games by luck. Its score looks better than it really is.

It caught us once

Adding coordinates to the screen looked like it fixed clicking on 2 practice games. On the test games it did nothing.

Even the test games aren't enough

Smarter random (E4) won on the 17 test games, then scored no better on the 110 hidden ones. Only the hidden games test whether an idea carries over.

The steps an experiment climbs

Each step is cheaper than the next, and each is allowed to say more. An experiment stops as soon as a step says stop. Steps 1–4 only ever use practice games.

1	Smoke a few games	Does the test program run from start to finish? Install, load the AI, play, save the results. Checks the setup only.	■ PROVES NOTHING
2	Diagnose 2 practice games	Why is something going wrong? A before-and-after on one suspected cause, such as the AI's clicking.	■ POINTS TO A FIX

3	Probe 2-3 practice games, 1 try	Does the new helper actually switch on? Replies can be read, the AI picks real buttons, nothing is broken.	A MECHANICS CHECK
4	Tune 8 practice games, 3 tries	Is this idea worth a real test? A go/stop rule, written before the run. This is where E2 and E3 stopped.	GO OR STOP ONLY
5	Test "held-out" · 16 test games	Does the idea beat the comparison? The only step whose numbers count as a result.	THE VERDICT
6	Competition 110 hidden games	Does it carry over to games nobody has seen? "Abandoned at tune" (E2) and "stopped at tune" (E3) mean the same thing: the rule said stop, so neither idea was ever run on the test games. Both stopped for the same reason — the AI wasted more moves with the new helper, not fewer.	DOES IT CARRY OVER

Which steps each experiment went through

EXPERIMENT	SMOKE	DIAGNOSE	PROBE	TUNE	TEST	COMPETITION
E0 · let the AI play	ran	ran	ran	—	ran	—
E1 · memory	—	—	ran	—	ran	—
E2 · explain the buttons	—	—	—	stopped	—	—
E3 · learn or win	—	—	—	stopped	—	—
E4 · smarter random	—	—	—	—	ran	Build 3

Steps were skipped when an earlier run had already covered them: E1 was built on E0's working setup, so it started at the probe. E4 uses no AI and runs on a laptop in minutes, so it went straight to the test games. Plain random was entered first as **Build 1** (0.07), the score every later entry had to beat.

Tries (seeds): how many times each game was played

Random choices and the AI's own replies both involve chance, so one run can be lucky. Each game is played several times, and each try is numbered — try 0, try 1, try 2 and so

on. The number sets the dice, so **try 3 always plays the same way**. That lets two players be compared try by try on identical luck, and lets one big run be split across two machines without changing a single move.

1-2

Diagnose and probe:
enough to see
whether it works at all

3

Tune on practice
games: enough for a
go/stop call

5

Test verdicts with the
AI: the project's
minimum for a result

20

Smarter random (E4):
wins are rare and a
laptop is cheap

Two honest caveats. The AI server doesn't always give exactly the same reply to the same prompt, so AI runs repeat almost exactly, not perfectly. And the project's rule says at least 5 tries; tune used 3 because it can only stop an idea, never prove one. The written rule doesn't say that exception out loud yet.

▼ Every run on Kaggle, and which step it was

KAGGLE RUN (ARC-AGI-3-...)	EXPERIMENT	STEP	GAMES × TRIES × MOVES
build-1-random	—	Competition	110 hidden · plain random, 80 moves → 0.07
e0-smoke	E0	Smoke	pipeline check
e0-diag	E0	Diagnose	dc22, ls20 × 1 × 80 · the “before”
e0-diag2	E0	Diagnose	dc22, ls20 × 2 × 80 · with coordinates
e0-gemma4-e4b	E0	Test	17 × 5 × 80 · small model
e0-qwen3-8b	E0	Test	17 × 5 × 80 · setup failure, not a result
e0-probe-gemma4-31b	E0	Probe	ls20, tu93 × 1 × 20 · big model
e0-gemma4-31b-e012	E0	Test	17 × tries 0-2 × 80
e0-gemma4-31b-e34	E0	Test	17 × tries 3-4 × 80 · one run split in two to fit Kaggle's 9-hour limit
e1-probe-e4b	E1	Probe	dc22, sc25, tu93 × 1 × 80
e1-heldout-e4b	E1	Test	17 × 5 × 80 · compared against E0's run
e2-tune-e4b	E2	Tune → stop	8 × 3 × 80 · six conditions
e3-tune-e4b	E3	Tune → stop	8 × 3 × 80 · version 1 crashed and played nothing

KAGGLE RUN (ARC-AGI-3-...)	EXPERIMENT	STEP	GAMES × TRIES × MOVES
----------------------------	------------	------	-----------------------

build-3-random-prune	E4	Competition	110 hidden · smarter random, 300 moves → 0.07
----------------------	----	-------------	---

E4's test run isn't listed: it ran on a laptop, 17 games × 20 tries at 80 and 300 moves.

HONEST CORRECTION

We found a bug in our own measuring — and fixed it

Our test program reused the same game between tries. After one real win, later tries started ahead and were counted as extra wins. We spotted it because some levels looked like they were cleared in a single move, which is impossible. **Random play had looked about three times better than it really was.** We fixed it, re-ran what it touched, and corrected every number. None of the AI results changed, and no conclusion flipped.

THE BIG PICTURE

Four things to take away

- **These puzzles are hard for small AI.** In our setup it never beat random button-pressing.
- **The main problem is wasted clicks** on things that do nothing. Blocking them helps exploration — but exploring is not the same as solving.
- **Test on games you haven't tuned on.** Our best trick worked on 17 games and vanished on 110 new ones.
- **More instructions didn't help this AI.** A button list (E2) and move labels (E3) both made it waste more moves.

WHERE THIS LEADS

From five experiments to the next two

Each experiment left one lesson, and each lesson set up the next question. The last one points to a gap that smarter random can't fill. Before trying to fill it, one cheap step makes sure the measurements can be trusted. Then there are two ways to try.

E0 · FROZEN MODEL

Can the AI beat random?

No. Its clicks often do nothing.

Letting the AI choose directly isn't enough

E1 · MEMORY

Does remembering failures help?

Partly. Blocking dead moves helps exploring; clears only match random.

Memory helps search, not understanding

E2 · BUTTON CHECK

Does explaining the buttons help?

No. Wasted moves go up.

Having the information isn't the same as using it well

E3 · LEARN OR WIN

Can the AI separate learning moves from winning moves?

Stopped at practice. Wasted moves go up; labels mostly ignored.

Labels and a diary don't give the AI a way to experiment

E4 · SMARTER RANDOM

How far can a simple rule go?

Doubles clears on test games; the hidden score doesn't move.

Simple search works on known games but doesn't carry over

THE GAP AFTER E4

Skipping dead moves can't work out how a game works

It can't infer the rules, the goal, cause and effect, or a plan.

First, make sure the measurements can be trusted

P1 · GROUNDWORK, FIRST · PLANNED

Ignore on-screen clocks, and add the safety checks

Many games show a clock that changes every move, so a useless move looks useful. Fix that “did anything change?” check, then see whether smarter random clears more. Laptop only, under an hour. Also builds the checks against another replay bug.

Can slow, careful reasoning fill the gap?

Or is a better fast choice of button enough?

E5 · DUCK HARNESS · PLANNED

Investigate on purpose

Inspect → form a guess → test it → update a model of the game → plan.

E6 · JEV · PLANNED

Fast learned choices

Can it rank useful buttons better than smarter random and simple statistics?

THEN DECIDE

Do E5 and E6 each add something the other doesn't?

Yes: each helps in different situations

No: one or both add nothing distinct

E7 · HYBRID · PLANNED

Fast by default, slow when stuck

Jev handles routine moves; Duck takes over when things are new, uncertain or keep failing.

OTHERWISE

Don't build the hybrid

Keep whichever method worked on its own, or rethink the design.

Dashed boxes are plans. None of P1, E5 or E6 has a written pass/fail rule yet, and none will run until it does. P1 comes first because every later number depends on the “did anything change?” check. E5 and E6 must beat the best simple method available when they run — smarter random at least.

WORDS USED ON THIS PAGE

A small glossary

Level	Each game has several stages. Clearing one is the main thing we counted.
Random baseline	A player that presses random allowed buttons. The bar every idea had to beat.
Frozen model	An AI model used as-is, never retrained. Only the program around it changed.
Helper (harness)	The program wrapped around the AI: it can block moves, add notes, or test buttons.
Practice vs test games	8 “tune” games for trying settings, 17 “held-out” games kept unseen for the real verdict (16 count; r11l is set aside). The competition adds 110 hidden games.
Stage	A step an experiment passes through: smoke, diagnose, probe, tune, test, competition. “Stopped at tune” means it never reached the test games.
Tries (seeds)	Numbered repeat runs. The number fixes the randomness, so each try replays exactly and two players can be compared on identical luck.
Exploration	How many different screens a player sees per 100 moves. More usually means less stuck.

ARC-AGI-3 research project, September 2026 · about 20 GPU-hours on Kaggle plus a laptop CPU · every rule, run and correction is kept in the project’s experiment register.